

FROM
ZERO TO
PRODUCTION

Redis Production Handbook

A PRACTICAL GUIDE FOR JUNIOR ENGINEERS

LEARN | CONFIGURE | SECURE | SCALE | MONITOR | TROUBLESHOOT



redis

FAST
RELIABLE
SCALABLE
IN-MEMORY DATA
FOR MODERN
APPLICATIONS

 **CACHING**  **DATA STRUCTURES**  **SECURITY**  **REPLICATION**  **HIGH AVAILABILITY**  **MONITORING**  **TROUBLESHOOTING**

REAL-WORLD EXAMPLES | PRACTICAL COMMANDS | PRODUCTION BEST PRACTICES

- ✓ 20-PAGE JUNIOR ENGINEER CURRICULUM
- ✓ HANDS-ON EXAMPLES
- ✓ TROUBLESHOOTING SCENARIOS
- ✓ PRODUCTION CONFIGURATIONS
- ✓ SECURITY AND BEST PRACTICES
- ✓ DEPLOYMENT WITH DOCKER AND KUBERNETES
- ✓ MONITORING AND ALERTING
- ✓ BACKUP AND RECOVERY

BUILD
REAL SKILLS
FOR REAL
SYSTEMS

PRACTICE
DEPLOY
OPERATE
LIKE A PRO

Redis for Junior Engineers

Understand what Redis is, why it is fast, where it fits and how engineers use it.

1. WHAT IS REDIS?

- Redis is an open source, **in-memory data store**.
- It stores data in RAM (memory) instead of disk, which makes it very fast.
- It supports different data structures like strings, hashes, lists, sets, sorted sets, and more.
- Redis is often used as a cache, but it can also be used as a message broker, session store, real-time data store and more.

2. IN-MEMORY DATA STORE (EXPLAINED SIMPLY)

- Most databases store data on disk (like a hard drive).
- Redis keeps data in memory (RAM).
- Memory is much faster than disk, so Redis can read and write data in milliseconds.
- Because the data is in memory, Redis is ideal for applications that need **speed, low latency and real-time access**.
- Redis can also save data to disk if you configure persistence, but its strength is speed.

3. REDIS VS TRADITIONAL DATABASE

Feature	Redis	Traditional Database
Storage	In-memory (RAM)	On disk (SSD/HDD)
Speed	Very fast (microseconds)	Slower (milliseconds)
Data types	Multiple (string, hash, list, set, etc.)	Usually tables/rows
Use case	Caching, real-time, sessions, queues	Long-term storage, complex queries
Persistence	Optional (RDB/AOF)	Always persistent
Best for	Speed and high throughput	Durability and complex data

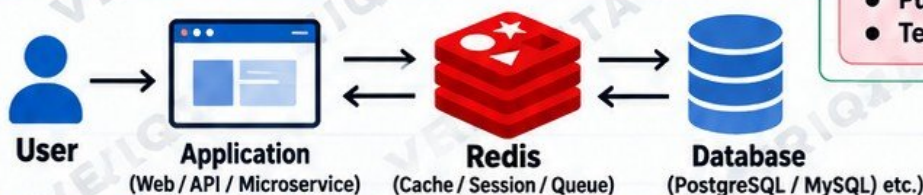
4. WHY REDIS IS FAST?

- Stores data in RAM instead of disk.
- Simple and efficient data structures.
- Single-threaded, event-driven architecture (handles many connections efficiently).
- Minimal overhead and optimized for speed.
- Can keep frequently used data in memory to avoid slow disk access.
- Built for **high throughput and low latency**.

5. COMMON PRODUCTION USES

- Caching frequently accessed data
- User session storage
- Real-time leaderboards and counters
- Message queues and background jobs
- Rate limiting (API protection)
- Real-time analytics and metrics
- Pub/Sub for real-time communication
- Temporary storage for distributed systems

6. WHERE REDIS SITS IN AN APPLICATION ARCHITECTURE



The application checks Redis first. If the data is not in Redis (cache miss), it retrieves the data from the **database** and can **store it in Redis** for future requests.

7. DEVOPS CONNECTION

- Infrastructure engineers deploy and manage Redis on servers, in Docker or Kubernetes.
- They configure memory, persistence, security, monitoring and high availability.
- They troubleshoot performance issues, memory usage and connection problems.
- Redis is commonly used in cloud environments (AWS ElastiCache, Azure Cache for Redis, Google Memorystore).
- Understanding Redis helps DevOps, SRE and platform engineers support **reliable, scalable and high-performance systems**.



JUNIOR ENGINEER TIP

Redis is not just a cache. It is a **high-performance building block** used in many modern applications.

How Redis Works

Understand the internal workings of Redis and how applications communicate with it.

1. MEMORY-FIRST ARCHITECTURE

- Redis stores data in memory (RAM) instead of disk.
- This allows very fast read and write operations (microseconds).
- It can also **persist** data to disk (RDB or AOF), but its primary strength is in-memory access.
- Memory-first design makes Redis ideal for high-performance and real-time use cases.

2. KEYS AND VALUES

- Redis stores data as **key-value pairs**.
- A key is a unique name (like a label).
- A value can be a string, hash, list, set, sorted set, stream or other data structure.
- Keys are case sensitive.
- Examples:
`SET user:1001 "John"`
`SET product:1 "Laptop"`
`GET user:1001`

3. REDIS SERVER AND CLIENTS

- Redis runs as a **server** process (redis-server).
- Clients connect to the server using a network connection (usually TCP).
- Clients can be applications, CLI tools (redis-cli), programming libraries or other services.
- Many clients can connect at the same time.

4. COMMANDS AND RESPONSES

- Clients send commands to the Redis server.
- The server executes the command and returns a response.
- Redis uses a simple text-based protocol called RESP (Redis Serialization Protocol).
- Example:
`SET name "VeriQTA" → OK`
`GET name → "VeriQTA"`
`DEL name → (integer) 1`

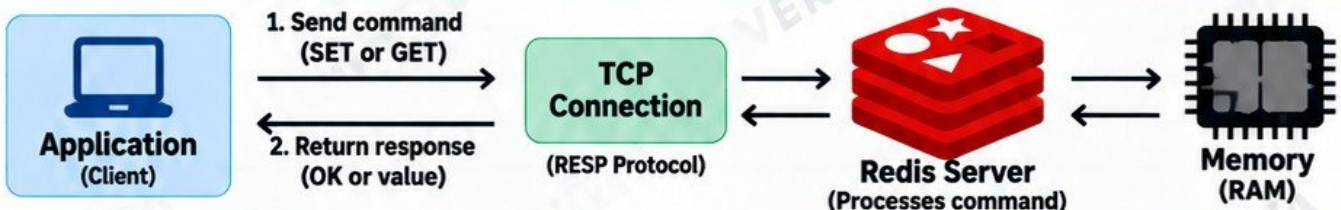
5. SINGLE-THREADED COMMAND EXECUTION

- Redis processes commands using a single thread.
- It handles many connections efficiently using an event-driven model (not one thread per client).
- This avoids the overhead of multi-threading and keeps performance high for most use cases.
- Long-running or blocking commands can affect other clients, so they should be avoided.

6. NETWORK CONNECTIONS

- Clients connect to Redis over **TCP** (default port 6379).
- Communication happens using the RESP protocol.
- Each command is sent over the network and executed by the Redis server.
- Network latency is usually very low when the application is in the same network (e.g. same VPC or cluster).
- Use TLS and firewalls in production for security.

7. WHAT HAPPENS WHEN AN APPLICATION RUNS GET OR SET?



- When you run SET, Redis stores the key and value in memory and returns OK.
- When you run GET, Redis looks up the key in memory and returns the value.
- This usually happens in microseconds, which is why Redis is very fast.

8. BEGINNER MISTAKE



Thinking Redis is just a cache.

- Redis is commonly used as a cache, but it can also be used for session storage, real-time data, message queues, leaderboards, rate limiting and more.
- Treating it as only a cache limits your understanding and usage.
- Redis is a versatile in-memory data store, not just a simple cache.

Installing and Running Redis

Get Redis installed, start the server, connect using redis-cli and run your first commands.

1. LOCAL INSTALLATION CONCEPTS

- Redis can be installed on Linux, macOS or Windows.
- For learning, we usually install it on a local machine or a cloud server (e.g. Ubuntu).
- In production, Redis is often deployed on dedicated servers, in containers (Docker) or managed services (AWS ElastiCache, etc.).
- Redis is open source and easy to install.

3. STARTING REDIS

After installation, start the Redis server:

```
sudo systemctl start redis-server
```

Check status:

```
sudo systemctl status redis-server
```

✓ If Redis is running, you will see it active (running).

2. INSTALLING REDIS (LINUX EXAMPLE)

On Ubuntu/Debian, you can install Redis using:

```
sudo apt update
sudo apt install redis-server
```

On macOS (using Homebrew):

```
brew install redis
```

On Windows: Use WSL (Ubuntu) or download from the official website.

4. USING REDIS-CLI

Connect to Redis using the command line client:

```
redis-cli
```

If Redis is running locally, you should see:

```
127.0.0.1:6379>
```

This means you are connected to the Redis server.

5. BASIC REDIS COMMANDS

Once connected, try these commands:

Command	What It Does	Example
PING	Checks if Redis is alive	PING
SET	Stores a key and value	SET name "VeriQTA"
GET	Retrieves a value	GET name
DEL	Deletes a key	DEL name
INFO	Shows server information	INFO

6. EXAMPLE SESSION

Here is a sample redis-cli session:

```
127.0.0.1:6379> PING
PONG
127.0.0.1:6379> SET name "VeriQTA"
OK
127.0.0.1:6379> GET name
"VeriQTA"
127.0.0.1:6379> DEL name
(integer) 1
127.0.0.1:6379> INFO
# Server
redis_version:7.2.4
...
```

7. PRACTICE LAB



- 1 Install Redis on your machine (or an Ubuntu server).
- 2 Start the Redis server.
- 3 Connect using redis-cli.
- 4 Run PING, SET, GET, DEL and INFO.
- 5 Take a screenshot of your working session.
- 6 Try creating your own key and value.
- 7 Delete the key and confirm it is gone.

8. IMPORTANT NOTES



- By default, Redis runs on port 6379.
- Keep Redis secure. Do not expose it to the public internet. Use authentication and firewalls in real environments.
- In production, use proper configuration files.
- Always stop the redis-cli session with:

```
127.0.0.1:6379> exit
```

Redis Keys, Values, and Expiration

Learn how Redis keys work, use common commands and understand why expiration matters.

1. HOW REDIS KEYS WORK

- A key is a unique name used to store a value in Redis.
- Keys are **case sensitive** (user and User are different keys).
- A key can hold a string, hash, list, set, sorted set or other data type.
- Redis keys behave like files names in a system, but they live in memory.
- Each key has exactly one value at a time.

2. KEY NAMING CONVENTIONS

- Use clear and descriptive names.
- Use a consistent format (e.g. app:feature:id).
- Use colons (:) to create a namespace structure.
- Keep keys short but meaningful.
- Avoid spaces and special characters.
- Use lowercase for consistency.
- Examples:


```
user:1001
product:2005
session:abc123
cache:homepage
rate_limit:user:1001
```

3. COMMON REDIS KEY COMMANDS

Command	What It Does	Example
SET	Stores a key and value	SET user:1001 "John"
GET	Retrieves the value of a key	GET user:1001
EXISTS	Checks if a key exists (returns 1 or 0)	EXISTS user:1001
DEL	Deletes a key (returns number deleted)	DEL user:1001
EXPIRE	Sets a time to live (TTL) in seconds	EXPIRE user:1001 3600
TTL	Checks remaining time to live (in seconds)	TTL user:1001

4. EXPIRING VS PERSISTENT KEYS

- Persistent keys have no expiration. They stay in Redis until manually deleted.
- Expiring keys automatically disappear after the TTL (time to live) expires.
- Example of a persistent key:


```
SET config:app "production"
```
- Example of an expiring key (1 hour):


```
SET session:1001 "data"
EXPIRE session:1001 3600
```

5. WHY EXPIRATION MATTERS

- Redis stores data in memory, which is limited.
- Without expiration, unused keys can fill up memory.
- Expiration helps keep Redis clean and efficient.
- It prevents stale data from being served.
- It reduces the risk of memory issues and eviction.
- It is essential for **caches, sessions and temporary data**.

6. PRACTICAL EXAMPLE

Set a key, check it, set expiration and verify:

```
# Set a key
SET user:1001 "John"
# Get the key
GET user:1001
# Check if it exists
EXISTS user:1001
# Set expiration (60 seconds)
EXPIRE user:1001 60
# Check remaining time
TTL user:1001
# Delete the key
DEL user:1001
```

Expected output:

```
OK
"John"
1
1
60 (or less)
1
```

7. BEGINNER MISTAKE



Creating cache keys without TTLs.

- Beginners often store data in Redis without setting an expiration time.
- This can lead to unused keys staying in memory forever.
- It wastes memory and can cause performance issues or evictions.
- **Always set an appropriate TTL for cache data, sessions and temporary information.**

8. JUNIOR ENGINEER TIP



If a key does not need to live forever, always set a TTL. It keeps Redis healthy, fast and reliable.

Redis Data Types

Understand the core Redis data structures, when to use them and see practical examples.

1. STRINGS

abc

- Stores a single value (string, number or binary data).
- Most commonly used data type.
- Good for simple key-value pairs, counters and small data.

Example:

```
SET user:1001 "John"
GET user:1001
```

Use cases:

- User sessions
- Configuration values
- Counters (INCR)

2. HASHES



- Stores multiple fields and values under one key.
- Like a JSON object or a database row.
- Good for structured data.

Example:

```
HSET user:1001 name "John"
HSET user:1001 age "25"
HGET user:1001 name
HGETALL user:1001
```

Use cases:

- User profiles
- Product details
- Application settings

3. LISTS



- Stores an ordered list of values.
- Supports push and pop from both ends.
- Good for queues and recent activity feeds.

Example:

```
LPUSH notifications "msg1"
LPUSH notifications "msg2"
LRANGE notifications 0 -1
```

Use cases:

- Task queues
- Background jobs
- Recent activity feeds

4. SETS



- Stores unique values (no duplicates).
- Unordered collection.
- Useful for membership checks and unique lists.

```
SADD user:1001:roles "admin"
SADD user:1001:roles "dev"
SMEMBERS user:1001:roles
```

Use cases:

- User roles and permissions
- Tags and categories
- Unique visitors

5. SORTED SETS (ZSETS)



- Stores unique values with a score for sorting.
- Values are ordered by score (low to high).
- Great for leaderboards, rankings and time-based data.

```
ZADD leaderboard 100 "user1"
ZADD leaderboard 200 "user2"
ZRANGE leaderboard 0 -1
WITHSCORES
```

Use cases:

- Leaderboards
- Ranking systems
- Rate limiting
- Time-based events

6. STREAMS



- Stores a sequence of messages (events).
- Each message has a unique ID, fields and values.
- Designed for real-time data and event processing.

```
XADD logs * level "info"
message "user login"
XRANGE logs - +
XREAD COUNT 1 STREAMS logs $
```

Use cases:

- Application logs
- Real-time event processing
- Message queues
- Undersrviccd communication

7. WHEN TO USE WHICH?

Data Type	Best For	Example Use Case
String	Simple key-value	User session token
Hash	Structured data	User profile
List	Ordered collection	Task queue
Set	Unique values	User roles
Sorted Set	Ordered by score	Leaderboard
Stream	Event data	Application logs

8. PRODUCTION AWARENESS



Choosing the wrong data type can:

- Increase memory usage.
- Make commands slower.
- Cause higher CPU usage.
- Make your application harder to maintain.
- Use the right data type for the problem.
- Monitor memory usage (INFO memory).
- Avoid storing very large values in Redis.
- Understand data size, expiration and access patterns.

9. KEY TAKEAWAYS

- ✓ Redis supports multiple powerful data types.
- ✓ Each data type solves different problems.
- ✓ Use the right data type for performance and efficiency.
- ✓ Understand your application's access patterns.
- ✓ Monitor memory and optimize as your data grows.

Redis as an Application Cache

Use Redis to make your applications faster, reduce database load and improve user experience.

1. WHAT CACHING SOLVES

- Caching stores frequently accessed data in memory for faster access.
- It reduces database load, improves response time and handles more users with the same resources.
- Instead of querying the database every time, the application checks Redis first.
- Ideal for data that does not change often or can tolerate short periods of staleness.

2. CACHE HIT vs CACHE MISS



Cache Hit

- Data is found in Redis.
- Returned immediately (fast).
- No database query required.



Cache Miss

- Data is not in Redis.
- Application queries the database.
- Data is stored in Redis for next time.

3. CACHE-ASIDE PATTERN

- 1 Application checks Redis for the data.
- 2 If found (cache hit), return it.
- 3 If not found (cache miss), query the database.
- 4 Store the data in Redis with a TTL (expiration time).
- 5 Return the data to the user.

4. TTL STRATEGY



- Always set a TTL (time to live) for cached data.
- Choose a TTL based on how often the data changes.
- Short TTL for frequently changing data.
- Longer TTL for stable data.
- Helps prevent stale data and saves memory.

Example:

```
SET product:1001 "data" EX 3600
(Expire in 1 hour)
```

5. CACHE INVALIDATION



- Remove or update cached data when the underlying data changes.
- Can be done by:
 - Deleting the key (DEL)
 - Updating the key
 - Using a short TTL
 - Event-based invalidation (e.g. after a database update)
- Without invalidation, users may see stale data.

6. STALE DATA



- Stale data is old data that is still in the cache after the original data has changed.
- Can lead to users seeing outdated information.
- Use TTL and proper invalidation to reduce staleness.

7. CACHE WARMING



- Preload important data into Redis before it is requested.
- Helps avoid cache misses for frequently accessed data.
- Useful after a deployment or when cache has been cleared.
- Common for product lists, configuration data or popular content.

8. CACHE STAMPEDE



- Happens when many users request the same data at the same time and the cache is empty.
- This can overload the database.
- Prevent it by:
 - Using a short-lived lock
 - Staggering TTLs
 - Using cache warming
 - Serving stale data while refreshing in the background

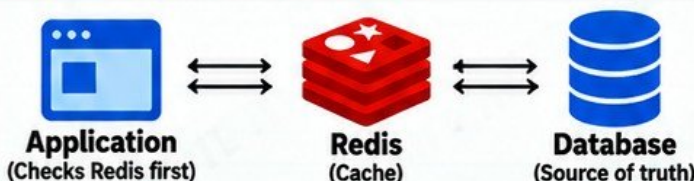
9. REAL WORLD EXAMPLE



E-commerce Product Page

- Product details (name, price, description, image) are requested thousands of times.
- Store this data in Redis with a TTL (e.g. 30 minutes).
- When the product is updated (e.g. price change), invalidate the cache key.
- Result: faster page loads and less load on the database.

10. ARCHITECTURE DIAGRAM



11. KEY TAKEAWAYS

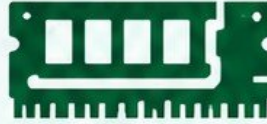
- ✓ Caching makes applications faster and more scalable. Use the cache-aside pattern.
- ✓ Always set a TTL.
- ✓ Invalidate cache when data changes.
- ✓ Be aware of stale data and cache stampedes.
- ✓ Cache frequently accessed and stable data.
- ✓ Monitor cache hit rate and database load.

Redis Memory Management

Understand how Redis uses memory, configure limits, choose the right eviction policy and avoid common problems.

1. WHY MEMORY IS REDIS'S CRITICAL RESOURCE

- Redis stores all data in memory (RAM) for very fast access.
- Memory is limited and more expensive than disk.
- If memory is not managed properly, Redis can run out of memory, causing errors, evictions or service disruption.
- Good memory management keeps Redis stable, fast and reliable in production.



Memory = Performance
Memory = Cost
Memory must be managed

2. MAXMEMORY

- `maxmemory` sets the maximum amount of memory Redis can use.
- When this limit is reached, Redis will either evict keys (based on policy) or return an error.
- If not set, Redis can try to use all available memory, which is risky.

```
# In redis.conf
maxmemory 2gb
maxmemory-policy allkeys-lru
```

3. MEMORY USAGE

- You can check overall memory usage and detailed memory information using the `INFO` command.
- This helps you understand how much memory is used, what data is using memory and how Redis is behaving.

```
127.0.0.1:6379> INFO memory
# Key sections in the output:
used_memory:104857600
used_memory_human:100.00M
used_memory_rss:120586240
maxmemory:2147483648
mem_fragmentation_ratio:1.15
```

4. MEMORY USAGE OF A KEY

- Use the `MEMORY USAGE` command to check how much memory a specific key is using.
- Useful for finding large keys that could cause memory issues.

```
127.0.0.1:6379> SET user:1001 "John"
OK
127.0.0.1:6379> MEMORY USAGE user:1001
56
```

5. EVICTION EXPLAINED

- When Redis reaches the `maxmemory` limit, it can evict keys based on the configured policy.
- Eviction removes keys to free up memory for new data.
- If eviction is not allowed (`noeviction`), Redis will return an error when memory is full.



Eviction helps Redis stay available, but you must choose the right policy for your use case.

6. NOEVICTION POLICY

- With `noeviction`, Redis does not remove any keys.
- When memory is full, write commands (e.g. `SET`, `HSET`) will return an error.
- This is useful when you cannot afford to lose any data (e.g. persistent cache or session store).

```
127.0.0.1:6379> CONFIG SET maxmemory-policy noeviction
OK
```

7. LRU AND LFU CONCEPTS

- LRU (Least Recently Used):** evicts keys that have not been used for the longest time.
- LFU (Least Frequently Used):** evicts keys that are used the least often.
- Both help keep frequently used data in memory.
- Choose LRU for recency-based workloads and LFU for frequency-based workloads.

8. ALLKEYS VS VOLATILE POLICIES

Policy	Applies To	Description
<code>allkeys-lru</code>	All keys	Evict least recently used keys.
<code>allkeys-lfu</code>	All keys	Evict least frequently used keys.
<code>volatile-lru</code>	Only keys with an expiration	Evict least recently used keys (with TTL).
<code>volatile-lfu</code>	Only keys with an expiration	Evict least frequently used keys (with TTL).
<code>volatile-ttl</code>	Only keys with an expiration	Evict keys with the shortest TTL.
<code>noeviction</code>	None	Do not evict keys.

9. WHAT HAPPENS WHEN REDIS REACHES ITS LIMIT?

- If a policy allows eviction, Redis will remove keys based on the policy.
- If `noeviction` is set, write commands will fail with an error like:

```
(error) OOM command not allowed when used memory > 'maxmemory'
```

- Applications may experience errors, timeouts or unexpected behaviour.
- You must monitor memory usage and plan for growth.

10. PRODUCTION AWARENESS



- Never treat Redis memory as unlimited.
- Set an appropriate `maxmemory` limit.
- Choose the right eviction policy for your workload.
- Monitor memory usage, evictions and key sizes.
- Remove unused keys and set TTLs.
- Test your configuration in a staging environment.
- Plan for memory growth as your application scales.

Redis Persistence, RDB and AOF

Learn how Redis saves data to disk, the differences between RDB and AOF, and how to choose the right approach.

1. WHY PERSISTENCE IS NEEDED

- Redis stores data in memory (RAM) for very fast access.
- Memory is volatile. If the server restarts or crashes, all data is lost unless persistence is enabled.
- Persistence allows Redis to save data to disk so it can be recovered after a restart.
- It is important for critical data, session data, queues and any data you cannot afford to lose.



In-memory is fast, but not permanent. Use persistence when data matters.

2. RDB SNAPSHOTS

- RDB (Redis Database) creates a point-in-time snapshot of your data and saves it to a file (dump.rdb).
- It is fast, compact and uses less disk space.
- It is created at configured intervals or manually.
- Good for disaster recovery and faster restarts.

Example redis.conf:

```
save 900 1 # after 900 sec
save 300 10 # after 300 sec
save 60 10000 # after 60 sec
# or manual snapshot
BGSAVE
```

3. AOF (APPEND ONLY FILE)

- AOF logs every write operation (command) to a file (appendonly.aof).
- It provides better durability and less data loss than RDB.
- Data is replayed from the AOF file during restart.
- The file can grow large, but it can be rewritten (AOF rewrite).
- You can configure how often writes are flushed to disk.

Example redis.conf:

```
appendonly yes
appendfsync everysec
# always (safer, slower)
# everysec (default)
# no (faster, risk of data loss)
```

4. RDB vs AOF COMPARISON

Feature	RDB	AOF
What it is	Point-in-time snapshot	Logs every write operation
Data loss risk	Higher (can lose recent data)	Lower (minimal data loss)
File size	Smaller	Larger
Performance	Faster	Slower
Best for	Backups, fast restarts	High durability
Default	Enabled (save rules)	Disabled (must enable)
File name	dump.rdb	appendonly.aof

5. COMBINING RDB AND AOF

- You can enable both RDB and AOF.
- RDB is used for fast restarts.
- AOF provides better durability for recent changes.
- On restart, Redis will use the AOF file (if enabled) because it is more up to date.



Using both gives you a good balance of performance and data safety.

6. PERSISTENCE TRADEOFFS

- RDB: better performance, smaller files, but can lose recent data.
- AOF: better durability, larger files, slightly slower.
- Both enabled: more disk usage, but safer.
- Choose based on your use case: cache (RDB), critical data (AOF) or both.



There is no single right answer. It depends on your performance, durability and storage requirements.

7. RESTART AND RECOVERY

- When Redis starts, it loads data from disk.
- If AOF is enabled, Redis will load the AOF file.
- If AOF is not enabled but RDB exists, it will load the RDB file.
- You can check the loading process in the logs.
- After recovery, verify that your data is available using redis-cli.

Check logs:

```
tail -f /var/log/redis/redis.log
```

Check keys:

```
redis-cli INFO persistence
```

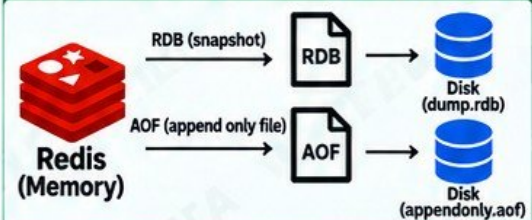
8. DATA-LOSS CONSIDERATIONS

- RDB can lose the most recent changes (based on save interval).
- AOF can lose the last few seconds of data (based on appendfsync).
- If both are disabled, all data is lost after a restart.
- Hardware failure, disk corruption or misconfiguration can also lead to data loss.
- Always test your recovery process in a non-production environment.



Persistence reduces the risk of data loss, but it is not a backup. Always have backups.

9. DIAGRAM: HOW PERSISTENCE WORKS



10. BEGINNER MISTAKE



Assuming Redis always preserves everything after a restart.

- By default, Redis may not have persistence enabled.
- Even with persistence, recent data can still be lost.
- Always understand your configuration and test recovery.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

KNOWLEDGE
BUILDS OPPORTUNITY



Redis Configuration for Production

Learn the important Redis configuration settings, understand what they do, and prepare Redis for production use.

1. REDIS.CONF



- The main configuration file for Redis.
- Controls network, memory, persistence, logging, security and more.
- Usually located at:
 - /etc/redis/redis.conf (Linux)
 - /usr/local/etc/redis.conf (Mac)

Always review and customize the configuration file before production.

2. BIND ADDRESSES

- Controls which network interfaces Redis listens on.
- Default is usually 127.0.0.1 (localhost only).
- In production, bind only to internal IP addresses or private networks.

```
# Example
bind 127.0.0.1
# or bind to a private IP
bind 10.0.0.10
```

3. PROTECTED MODE



- Enabled by default in recent Redis versions.
- Prevents external connections if no authentication is set.
- Helps protect against accidental exposure.

```
# Example
protected-mode yes
# Set to no only if you have
proper security controls
protected-mode no
```

4. PORTS



- Default Redis port is 6379.
- You can change the port to avoid conflicts or for security through obscurity (but not a replacement for real security).

```
# Example
port 6379
# Change to a different port
port 6380
```

5. MEMORY LIMITS



- Use maxmemory to set a memory limit (very important in production).
- Helps prevent the server from using all system memory.

```
# Example
maxmemory 2gb
# Use appropriate unit
# (kb, mb, gb)
maxmemory 4gb
```

6. EVICTION CONFIGURATION



- When memory is full, Redis can evict keys based on a policy.
- Common policies:
 - noeviction
 - allkeys-lru
 - volatile-lru
 - allkeys-lfu
 - volatile-lfu
 - volatile-ttl

```
# Example
maxmemory-policy allkeys-lru
```

7. PERSISTENCE SETTINGS



- Configure RDB and/or AOF based on your durability requirements.
- RDB: creates snapshots.
- AOF: logs every write operation.

```
# RDB example
save 900 1
save 300 10
save 60 10000
# AOF example
appendonly yes
appendfsync everysec
```

8. LOGGING



- Logs help with troubleshooting and monitoring.
- Set the log level and log file location.
- Common log levels: debug, verbose, notice, warning.

```
# Example
loglevel notice
logfile /var/log/redis/redis.log
```

9. TIMEOUTS



- Control how long Redis waits for idle connections.
- Helps free up resources.

```
# Example
timeout 300
# Close idle connections
after 300 seconds (5 minutes)
```

10. CONNECTION LIMITS



- Set a limit on the number of client connections.
- Prevents exhaustion of server resources.

```
# Example
maxclients 10000
# Adjust based on your environment
```

11. REVIEW DEFAULTS



- Default settings are designed for development, not production.
- You should review and adjust: network, security, memory, persistence, logging and limits.
- Every production environment has different requirements.



Do not rely on defaults.
Review, test and document
your configuration.

12. SECURITY REMINDER



- Never expose Redis to the public internet.
- Use firewalls, security groups and private networks.
- Enable authentication (requirepass) in production.
- Use TLS if needed.
- Regularly review access and monitor for unusual activity.



Never expose Redis casually
to the public internet.

Redis Security and Access Control

Protect your Redis data, control access and follow security best practices in production.

1. NETWORK ISOLATION



- Keep Redis in a private network (VPC, private subnet).
- Do not expose Redis to the public internet.
- Allow access only from trusted applications and administrators.
- Use security groups, firewalls and network ACLs to control access.

Best Practice:

Place Redis in a private subnet and restrict inbound traffic to only required IP addresses or security groups.

2. AUTHENTICATION



- Require authentication to prevent unauthorized access.
- Use a strong password.
- Configure using `requirepass` (in simple setups) or ACLs (in modern setups).

```
# In redis.conf
requirepass StrongPassword123
```

Never run Redis without authentication in production.

3. REDIS ACLs



- Redis Access Control Lists (ACLs) allow you to create multiple users with different permissions.
- Give each user only the commands they need.
- Helps enforce the principle of least privilege.

```
# Create a read-only user
ACL SETUSER readonly on >pass123
~* +get +info

# Create an admin user
ACL SETUSER admin on >admin123
~* +@all
```

4. USERS AND PERMISSIONS



- Create different users for different applications.
- Grant only required commands and key patterns.
- Avoid using a single admin user for everything.

User	Permissions	Use Case
app-user	+get +set +del	Application access
readonly	+get +info	Monitoring / read-only
admin	+@all	Administration only

5. TLS ENCRYPTION



- Use TLS to encrypt data in transit.
- Prevents eavesdropping and man-in-the-middle attacks.
- Especially important when Redis is accessed over networks or between cloud services.

```
# In redis.conf
port 0
tls-port 6379
tls-cert-file /path/to/redis.crt
tls-key-file /path/to/redis.key
```

6. LEAST PRIVILEGE



- Give users only the minimum commands and key access they need.
- Do not use broad permissions like `+@all` unless necessary.
- Limit access by key patterns using `~` prefix.

Example:

```
# Allow access only to app:* keys
ACL SETUSER app-user on >pass123
~app:* +get +set +del
```

7. SECRET MANAGEMENT



- Do not hardcode Redis passwords in code or scripts.
- Use a secrets manager such as AWS Secrets Manager, Azure Key Vault or HashiCorp Vault.
- Rotate secrets regularly.
- Restrict who can access the secrets in your cloud environment.

8. DANGEROUS COMMANDS



- Some commands can be dangerous if misused:

```
FLUSHALL # Deletes all data
FLUSHDB  # Deletes current DB
CONFIG  # Changes configuration
SHUTDOWN # Stops Redis server
KEYS *   # Can block the server
```

- Restrict or disable these commands for non-admin users.
- Use ACLs to block dangerous commands.

9. CLOUD SECURITY GROUPS AND FIREWALLS



- Use cloud security groups (AWS, Azure, GCP) to allow only trusted traffic.
- Restrict inbound access to specific IP addresses or internal networks.
- Monitor and review firewall rules regularly.
- Do not allow 0.0.0.0/0 (unrestricted access).

10. CREDENTIAL ROTATION



- Change passwords and tokens regularly.
- Update credentials in all applications.
- Automate rotation using your cloud provider or a secrets management tool.
- Test to make sure new credentials work before removing old ones.

SECURITY REMINDER



Redis should normally be reachable only by authorized workloads and administrators. Do not expose Redis casually to the public internet.

Redis Replication

Replicate your data, scale your reads and improve availability with Redis replication.

1. PRIMARY AND REPLICA ROLES



Primary
(Master)

- Primary (master): accepts write operations (SET, DEL, INCR, etc.).
- Replicas: maintain a copy of the primary's data.
- Replicas are read-only by default.
- You can have multiple replicas for a single primary.

Replication helps with high availability and read scaling.

2. ASYNCHROUS REPLICATION



- Redis replication is asynchronous.
- The primary writes data and then sends the changes to replicas.
- The primary does not wait for replicas to confirm the data has been received.
- This provides better performance but can lead to replication lag.

Asynchronous replication means there can be a small delay before data appears on replicas.

3. INITIAL SYNCHRONIZATION



- When a replica is first created, it performs a full synchronization.
- The primary creates an RDB snapshot and sends it to the replica.
- After the snapshot is loaded, ongoing changes are sent as a stream of commands.
- This process is called a full resynchronization.

4. REPLICATION LAG



- Replication lag is the delay between when a write happens on the primary and when it appears on the replica.
- Lag can be caused by:
 - Large write volume
 - Network latency
 - Slow replicas
 - Full resynchronization
 - Resource constraints

Use INFO replication to check the replication status and lag.

5. READ SCALING



- Replicas can handle read-only traffic.
- This helps distribute read load and improve application performance.
- Common in applications with many more reads than writes.
- Use a load balancer or application logic to route read requests to replicas.

6. FAILURES AND CONSISTENCY



- If a replica fails, the primary continues to operate.
- You can add a new replica and it will resynchronize.
- If the primary fails, you need a failover process (manual or automated with tools like Redis Sentinel or managed services).
- Because replication is asynchronous, the replica may not have the very latest writes (eventual consistency).

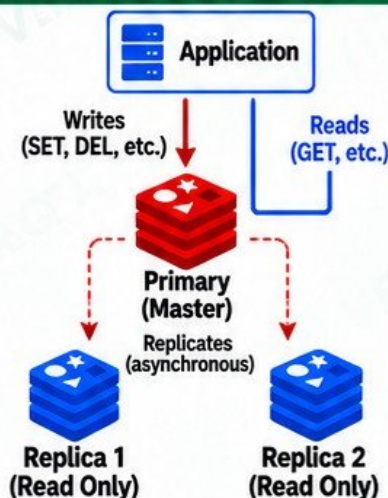
Replication improves availability but does not guarantee zero data loss in a failure.

7. INFO REPLICATION

Use the INFO replication command to check the replication status.

```
127.0.0.1:6379> INFO replication
# Replication
role:master
connected_slaves:2
slave0:ip=192.168.1.10,port=6379,
state=online,lag=0
slave1:ip=192.168.1.11,port=6379,
state=online,lag=1
master_repl_offset:123456789
repl_backlog_active:1
repl_backlog_size:1048576
repl_backlog_first_byte_offset:123000000
repl_backlog_histlen:456789
```

8. REPLICATION DIAGRAM



9. PRODUCTION AWARENESS



- Replication is not the same thing as backup.
- Replicas can have lag and may not contain the latest data.
- Use replication for high availability and read scaling.
- Use proper backup strategies (RDB, AOF or cloud backups) for disaster recovery.
- Monitor replication lag and replica health.

10. KEY TAKEAWAYS

- ✓ Primary handles writes, replicas handle reads.
- ✓ Replication is asynchronous.
- ✓ Initial sync uses an RDB snapshot.
- ✓ Replication lag is normal.
- ✓ Use replicas for read scaling.
- ✓ Replication is not a backup.
- ✓ Monitor using INFO replication.
- ✓ Plan for failover and consistency.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

KNOWLEDGE
BUILDS OPPORTUNITY



Redis Sentinel and High Availability

Use Redis Sentinel to monitor, detect failures and provide automatic failover for high availability.

1. WHY A SINGLE REDIS SERVER IS A FAILURE RISK



- If the only Redis server fails (crashes, hardware issue, network issue, maintenance), all applications lose access to data.
- This causes downtime, service disruptions and potential data loss (depending on persistence).
- High availability requires redundancy and automatic failover.

A single Redis server is a single point of failure (SPOF).

2. WHAT SENTINEL DOES



- Redis Sentinel is a high availability solution for Redis.
- It monitors Redis servers (primary and replicas).
- It automatically detects failures and performs failover.
- It promotes a replica to become the new primary.
- It notifies clients about the new primary so they can reconnect.

Sentinel does not store data. It only monitors, detects failures and manages failover.

3. MONITORING



- Sentinel continuously checks the health of the primary and replicas.
- It uses periodic PING messages.
- If a server does not respond within the configured time, it is marked as subjectively down (SDOWN).
- If multiple Sentinels agree, it is marked as objectively down (ODOWN).

```
# Example sentinel.conf
sentinel monitor mymaster
127.0.0.1 6379 2
# 2 = number of sentinels that
must agree to mark down
```

4. FAILURE DETECTION



- Each Sentinel monitors Redis instances.
- If a primary is not responding, it is marked as SDOWN (subjective down).
- When a quorum of Sentinels agree, it is marked as ODOWN (objective down).
- This reduces false positives caused by temporary network issues.

5. LEADER ELECTION



- When a primary is marked down, the Sentinels run a leader election process.
- One Sentinel becomes the leader.
- The leader is responsible for initiating the failover.
- This prevents multiple Sentinels from performing conflicting failovers.

6. AUTOMATIC FAILOVER



- The leader Sentinel selects the best replica to promote as the new primary.
- It reconfigures the other replicas to follow the new primary.
- It updates the configuration so clients can discover the new primary.
- Failover happens automatically without manual intervention.

7. REPLICA PROMOTION



- Sentinel chooses a suitable replica (based on replication offset, priority, etc.).
- The selected replica is promoted to primary.
- Other replicas are reconfigured to replicate from the new primary.
- The old primary (when it recovers) will be reconfigured as a replica.

The replica with the most up-to-date data is usually chosen.

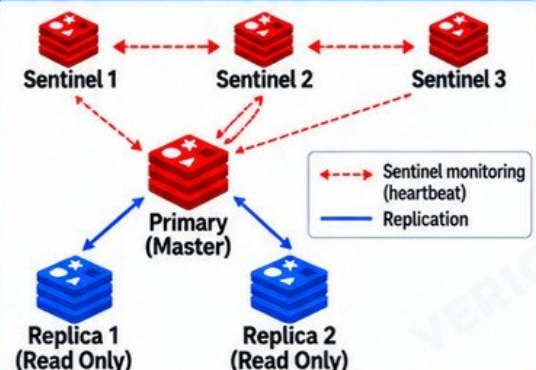
8. CLIENT DISCOVERY



- Clients can connect to Sentinels to discover the current primary.
- When a failover happens, clients are informed about the new primary.
- Clients should be configured to automatically reconnect (using Sentinel-aware clients like redis-py, Jedis, Lettuce, etc.).

Use Sentinel-aware clients in production so your application automatically finds the new primary.

9. ARCHITECTURE DIAGRAM



10. COMMON FAILURE



- After a failover, the application continues targeting the old primary.
- This happens when the client is not Sentinel-aware or does not refresh the primary address.
- Result: application errors, connection failures or writes going to a read-only replica.

11. KEY TAKEAWAYS

- ✓ Use Redis Sentinel for high availability.
- ✓ Sentinel monitors, detects failures and performs automatic failover.
- ✓ Use Sentinel-aware clients for automatic discovery.
- ✓ Test failover in a non-production environment.
- ✓ Replication is not the same as backup.

Redis Cluster and Horizontal Scaling

Scale your Redis deployment with sharding to handle larger datasets and higher traffic.

1. WHY REDIS CLUSTER EXISTS



- A single Redis server has memory and performance limits.
- Redis Cluster allows horizontal scaling by automatically splitting data across multiple nodes.
- It provides high availability and continues to work even if some nodes fail.

Use Redis Cluster when your data or traffic grows beyond a single server.

2. SHARDING



- Sharding means splitting your data into smaller parts and storing them on different nodes.
- In Redis Cluster, sharding is done automatically using hash slots.
- Each shard (node) is responsible for a subset of the data.

Sharding allows Redis to scale horizontally by adding more nodes as your data grows.

3. HASH SLOTS



- Redis Cluster has 16,384 hash slots (0 - 16383).
- Each key is hashed using CRC16 and assigned to a slot.
- Each node is responsible for a range of slots.
- The cluster automatically moves slots when nodes are added or removed.

Example:

```
Key: user:1001
CRC16(user:1001) = 3443
Slot: 3443
Stored on the node that owns slot 3443
```

4. NODES



- A Redis Cluster consists of multiple nodes.
- Each node can be a primary or a replica.
- Nodes communicate with each other to maintain the cluster state.

Minimum recommended:
At least 3 master nodes with replicas for high availability.

5. PRIMARIES AND REPLICAS



Primary
(Handles writes)



Replica
(Handles reads)

- Each shard has one primary and zero or more replicas.
- The primary node handles all write operations for its slots.
- Replicas maintain a copy of the data for high availability and read scaling.

6. KEY DISTRIBUTION



- Keys are automatically distributed across nodes based on their hash slot.
- You do not need to manually choose which node stores a key.
- The cluster balances data across nodes.

Example:

```
user:1001 → slot 3443 → Node 1
product:2001 → slot 8765 → Node 2
order:3001 → slot 12000 → Node 3
```

7. CLUSTER-AWARE CLIENTS



- Use a Redis client that understands Redis Cluster.
- It automatically routes commands to the correct node based on hash slots.
- Handles node changes, failover and redirection (MOVED and ASK responses).

Examples of cluster-aware clients:

- `redis-cli --cluster`
- `Jedis (cluster mode)`
- `Lettuce (cluster mode)`
- `redis-py (cluster mode)`

8. NODE FAILURES



- If a primary node fails, one of its replicas is automatically promoted.
- The cluster reassigns slots to the new primary.
- Clients are redirected to the new node.
- The cluster continues to operate without manual intervention.

Node failures are handled automatically, but there may be a short period of unavailability during failover.

9. MULTI-KEY OPERATION LIMITATIONS

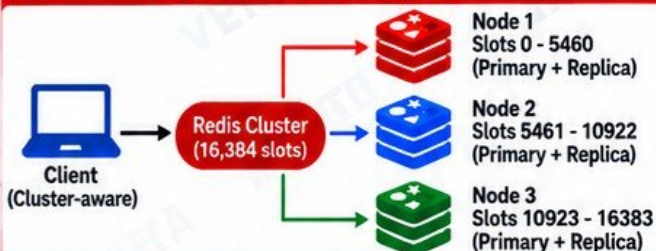


- Multi-key commands (e.g. MGET, MSET, DEL) only work if all keys are in the same hash slot.
- If keys are in different slots, the command will fail with a CROSSSLOT error.
- Use hash tags `{}` to force related keys into the same slot if needed.

Example with hash tags:

```
user:{1001}:name
user:{1001}:email
Both keys will be in the same slot.
```

10. CLUSTER ARCHITECTURE DIAGRAM



11. BEGINNER MISTAKE



Confusing Redis Cluster with Sentinel.

- Redis Cluster = horizontal scaling with sharding (multiple nodes, automatic slot management).
- Redis Sentinel = high availability for a single primary (automatic failover, no sharding).
- They solve different problems and are not the same.

12. KEY TAKEAWAYS

- ✓ Redis Cluster enables horizontal scaling with sharding.
- ✓ Data is split into 16,384 hash slots across multiple nodes.
- ✓ Each shard has a primary and replicas.
- ✓ Use cluster-aware clients.
- ✓ Multi-key commands only work for keys in the same slot.
- ✓ Cluster handles node failures automatically.
- ✓ Do not confuse Redis Cluster with Sentinel.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

KNOWLEDGE
BUILDS OPPORTUNITY



Redis with Docker and Kubernetes

Deploy and manage Redis using Docker and Kubernetes with the right configuration and storage.

1. RUNNING REDIS IN A CONTAINER



- Redis can run in a Docker container using the official image.
- Quick and easy for development and testing.
- For production, you must handle persistence, configuration, security and monitoring.

```
# Run Redis with Docker
docker run -d --name redis
-p 6379:6379 redis:7
```

2. PERSISTENT STORAGE CONSIDERATIONS



- Containers are ephemeral (data is lost when the container dies).
- Use Docker volumes or Kubernetes PersistentVolumes.
- Store Redis data in /data (by default).
- Ensure proper backup strategy for important data.

```
# Docker volume example
docker run -d --name redis
-v redis-data:/data
-p 6379:6379 redis:7
```

3. CONFIGURATION



- Use a custom redis.conf for production settings.
- Configure memory limits, persistence, security and logging.
- Mount the configuration file into the container or use a ConfigMap in Kubernetes.

```
# Docker: mount custom config
docker run -d --name redis
-v $(pwd)/redis.conf:/usr/local/etc/redis/redis.conf
-p 6379:6379 redis:7
redis-server /usr/local/etc/redis/redis.conf
```

4. SECRETS



- Do not hardcode passwords in your Dockerfile or manifests.
- Use environment variables, Docker secrets or Kubernetes Secrets.
- Keep secrets out of source control.

```
# Example: using environment variable
docker run -d --name redis
-e REDIS_PASSWORD=yourpassword
redis:7
```

5. KUBERNETES STATEFULSETS



- Use a StatefulSet for Redis in Kubernetes.
- StatefulSets provide stable network identity and persistent storage.
- Each pod has a unique identity (e.g. redis-0, redis-1).

```
# Example: StatefulSet (simplified)
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
spec:
  serviceName: redis
  replicas: 3
  selector:
    matchLabels:
      app: redis
```

6. SERVICES



- Use a Headless Service for StatefulSet (internal DNS).
- Use a ClusterIP service for client access within the cluster.
- Use a LoadBalancer or Ingress (if external access is required and secure).

```
# Headless service example
apiVersion: v1
kind: Service
metadata:
  name: redis
spec:
  clusterIP: None
  selector:
    app: redis
  ports:
    - port: 6379
      name: redis
```

7. PERSISTENT VOLUME CLAIMS



- Use PersistentVolumeClaims (PVCs) to store Redis data.
- Each pod in the StatefulSet gets its own PVC.
- Use a storage class appropriate for your environment (e.g. SSD for production).

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: redis-data
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 10Gi
```

8. RESOURCE REQUESTS AND LIMITS



- Set CPU and memory requests and limits.
- Prevents resource starvation and noisy neighbor issues.
- Redis is memory intensive, so plan your limits carefully.

```
resources:
  requests:
    memory: "512Mi"
    cpu: "250m"
  limits:
    memory: "1Gi"
    cpu: "500m"
```

9. HEALTH PROBES



- Use liveness and readiness probes to monitor Redis.
- Helps Kubernetes restart unhealthy pods and only send traffic to ready pods.

```
livenessProbe:
  exec:
    command: ["redis-cli", "ping"]
  initialDelaySeconds: 10
  periodSeconds: 10
readinessProbe:
  exec:
    command: ["redis-cli", "ping"]
  initialDelaySeconds: 5
  periodSeconds: 10
```

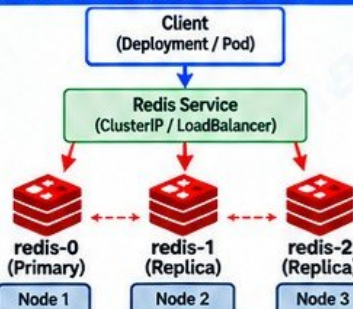
10. ANTI-AFFINITY CONCEPT



- Use pod anti-affinity to spread Redis pods across different nodes.
- Prevents all replicas from running on the same node.
- Improves availability during node failures.

```
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchLabels:
            app: redis
        topologyKey: "kubernetes.io/hostname"
```

11. EXAMPLE ARCHITECTURE



12. PRODUCTION AWARENESS



- A Redis container is disposable, production data may not be.
- Always use persistent storage.
- Plan for backups and disaster recovery.
- Use proper configuration, security, resource limits and monitoring.
- Test failover and pod rescheduling scenarios.

A container can be recreated.
Your data might not be.

13. KEY TAKEAWAYS

- Use containers for consistency, but persist your data.
- Configure Redis properly.
- Use Kubernetes StatefulSets for production.
- Set resource requests, limits and health probes.
- Spread pods across nodes using anti-affinity.
- Remember: a Redis container is disposable, production data may not be.

Redis Performance and Slow Commands

Understand what affects Redis performance, find slow commands and keep your Redis fast and reliable.

1. LATENCY VERSUS THROUGHPUT



- Latency is the time it takes to complete a single request (ms).
- Throughput is the number of requests Redis can handle per second.
- A system can have high throughput but still have high latency.
- Both are important for a good user experience.

Goal: Keep latency low and throughput high.

2. COMMAND COMPLEXITY



- Some Redis commands are $O(1)$ and very fast (e.g. GET, SET).
- Some commands are $O(n)$ and can be slow on large datasets (e.g. LRANGE, SMEMBERS, HGETALL).
- The bigger your data, the more time complex commands take.

Check command complexity in the Redis documentation before using a command in production.

3. LARGE KEYS



- Large keys consume more memory.
- They take longer to read, write and transfer over the network.
- Large keys can block the Redis event loop and impact performance.
- Avoid storing very large values (e.g. multi-MB strings) in a single key.

Tip: Keep your key values small and split large data when possible.

4. HOT KEYS



- A hot key is a key that receives a very high number of requests.
- Hot keys can create uneven load and become a performance bottleneck.
- They can increase CPU usage and latency.
- Use key distribution and caching strategies to avoid hot keys.

Monitor key access patterns to identify and handle hot keys early.

5. BLOCKING COMMANDS



- Some commands can block Redis and affect all clients (e.g. KEYS, FLUSHALL, LRANGE on large lists).
- Blocking commands run in the main Redis thread.
- While a command is running, other requests must wait.
- Avoid blocking commands in production environments.

Use non-blocking alternatives like SCAN instead of KEYS.

6. SLOWLOG



- Redis SLOWLOG records commands that take longer than a specified time (default 10ms).
- It helps you find which commands are slow.
- Useful for troubleshooting performance issues.

```
# View slow log
SLOWLOG GET 10
# Check slow log length
SLOWLOG LEN
# Set slow log threshold (microseconds)
CONFIG SET slowlog-log-slower-than 10000
```

7. LATENCY



- The LATENCY command provides latency metrics for different events in Redis.
- Helps identify spikes and performance issues.
- Useful for ongoing monitoring and troubleshooting.

```
# View latency events
LATENCY LATEST
# View latency history for an event
LATENCY HISTORY command 10
# Reset latency data
LATENCY RESET
```

8. INFO COMMANDSTATS



- Provides per-command statistics including calls, CPU time and average time.
- Helps identify which commands are used most and which ones are slow.
- Useful for capacity planning and optimization.

```
# View command statistics
INFO commandstats
# Example output (truncated)
cmdstat_get:calls=1000,usec=5000,usec_per_call=5
cmdstat_set:calls=800,usec=4000,usec_per_call=5
```

9. NETWORK LATENCY



- Network latency between your application and Redis affects overall performance.
- High latency can be caused by distance, network congestion or poor infrastructure.
- Keep your application close to Redis (same VPC or region when possible).

Use monitoring to track network latency and identify issues.

10. CPU SATURATION



- Redis is single-threaded, so high CPU usage can increase latency.
- CPU can be saturated by complex commands, hot keys or very high traffic.
- Monitor CPU usage and commandstats to find the cause.

If CPU is always high, review your workload, optimize commands or scale your architecture.

11. WHY KEYS * CAN BE DANGEROUS



- KEYS * scans the entire keyspace and blocks the Redis server.
- It can cause high latency and make Redis unresponsive.
- In production, use SCAN instead, which is non-blocking.

```
# Avoid this in production
KEYS *
# Use this instead
SCAN 0
```

12. TROUBLESHOOTING TIP



- Measure before changing configuration.
- Use SLOWLOG, LATENCY, INFO commandstats and monitoring tools.
- Identify the real bottleneck (first data, network, CPU or commands) before making changes.

Troubleshooting Tip:
Measure before changing configuration.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

KNOWLEDGE
BUILDS OPPORTUNITY



Redis Monitoring and Observability

Gain visibility into your Redis performance, health and capacity with the right metrics, tools and alerts.

1. WHAT ENGINEERS SHOULD MONITOR



- Memory utilization
- CPU usage
- Operations per second (OPS)
- Connections
- Cache hit ratio
- Evictions
- Expired keys
- Replication lag
- Latency
- Persistence failures

Monitoring helps you detect problems early, improve performance and keep Redis reliable in production.

2. MEMORY UTILIZATION



- Track `used_memory` and `used_memory_rss`.
- Monitor `memory_percentage` (how close you are to the limit).
- Watch for memory fragmentation.
- Set alerts before Redis reaches `maxmemory`.

High memory usage can lead to evictions, write failures or application errors.

3. CPU AND OPERATIONS



- Monitor CPU usage (system and Redis).
- Track `instantaneous_ops_per_sec` (OPS).
- A sudden drop in OPS may indicate a problem.
- High CPU can be caused by expensive commands, large keys or high traffic.

Sustained high CPU usage affects latency and overall performance.

4. CONNECTIONS



- Monitor total connections and `connected_clients`.
- Watch for unexpected spikes in connections.
- High connection counts can lead to memory and file descriptor exhaustion.

Set limits and alert on abnormal connection growth.

5. CACHE HIT RATIO



- Track `keyspace_hits` and `keyspace_misses`.
- A high hit ratio means your cache is working well.
- A low hit ratio may indicate poor cache design, wrong TTL or insufficient memory.

Monitor hit ratio trends over time to optimize performance.

6. EVICTIONS AND EXPIRED KEYS



- Monitor `evicted_keys` (keys removed due to memory pressure).
- Track `expired_keys` (keys removed after TTL).
- A high number of evictions may indicate your memory limit is too low.

Frequent evictions can cause cache misses and application latency.

7. REPLICATION LAG



- Monitor `master_repl_offset` and `slave_repl_offset`.
- Check replication lag (`seconds_behind_master`).
- High replication lag can affect lead consistency and failover time.

Alert if replication lag is higher than your acceptable threshold.

8. LATENCY



- Monitor command latency using the `LATENCY` command.
- Track `p50`, `p95` and `p99` latency for key operations.
- Identify slow commands and performance spikes.

High latency affects user experience. Investigate the root cause early.

9. PERSISTENCE FAILURES



- Monitor RDB and AOF persistence status.
- Check for last save time, AOF rewrite status and errors.
- Set alerts for failed or incomplete persistence operations.

Persistence failures can lead to data loss after a restart.

10. REDIS EXPORTER



- Redis Exporter collects Redis metrics and exposes them in a format that Prometheus can scrape.
- It provides metrics like memory, CPU, commands, connections, hits, evictions and more.
- Easy to deploy using Docker or Kubernetes.

Example (Docker):

```
docker run -d --name redis-exporter \
-p 9121:9121 \
oliver006/redis_exporter \
--redis.addr=redis://localhost:6379
```

11. PROMETHEUS AND GAFANA



- Prometheus scrapes metrics from Redis Exporter.
- Grafana visualizes the metrics with dashboards.
- Use pre-built dashboards or create your own.
- Monitor trends and set alerts for critical metrics.

Useful Resources:

- Redis Exporter (GitHub)
- Prometheus (prometheus.io)
- Grafana (grafana.com)
- Grafana Redis Dashboard (community dashboards)

12. ALERTING PRINCIPLES



- Set clear thresholds (e.g. memory > 80%, high latency, replication lag).
- Alert on trends, not just single spikes.
- Avoid alert fatigue by tuning thresholds.
- Use multiple channels (email, Slack, Telegram, etc.).
- Have a runbook for common alerts.

Good alerting helps you respond fast and prevent major incidents.

13. MONITORING ARCHITECTURE DIAGRAM



14. KEY TAKEAWAYS

- ✓ Monitor memory, CPU, OPS, connections and hit ratio.
- ✓ Track evictions, expired keys and replication lag.
- ✓ Use Redis Exporter with Prometheus and Grafana.
- ✓ Set meaningful alerts and have a runbook.
- ✓ Observability helps you keep Redis reliable and performant.

Redis Production Failure Scenarios

Learn how to troubleshoot common Redis problems using a structured approach.

1. CONNECTION REFUSED



- Symptom:** Cannot connect to Redis (e.g. Connection refused).
- Possible causes:** Redis not running, wrong host/port, firewall/security group.

Check:
`nc -zv <host> <port>`
`redis-cli -h <host> -p <port>`
`ss -tulnp | grep 6379`

Fix: Start Redis, check network and security groups, verify port.

Verify: `redis-cli ping`

2. AUTHENTICATION FAILURE



- Symptom:** NOAUTH Authentication required or invalid password.
- Possible causes:** Missing or wrong password, ACL misconfiguration.

Check:
`redis-cli -a <password> ping`
`ACL LIST`
`CONFIG GET requirepass`

Fix: Use correct password, update ACLs or requirepass configuration.

Verify: `redis-cli ping`

3. REDIS BECOMES SLOW



- Symptom:** High latency and slow responses.
- Possible causes:** Large keys, expensive commands, high CPU, memory pressure, network latency.

Check:
`SLOWLOG GET 10`
`INFO commandstats`
`INFO cpu`
`MONITOR` (use carefully)

Fix: Identify slow commands, optimize queries, remove hot keys, scale if needed.

Verify: Check latency again.

4. MEMORY SUDDENLY INCREASES



- Symptom:** Used memory increases rapidly.
- Possible causes:** Large keys, unexpected data growth, no expiration, memory leak in application.

Check:
`INFO memory`
`MEMORY USAGE <key>`
`SCAN 0 MATCH * COUNT 100` (key analysis)

Fix: Find and remove large/unused keys, set expiration, optimize application.

Verify: Monitor memory usage.

5. OOM COMMAND NOT ALLOWED



- Symptom:** (error) OOM command not allowed when used memory > 'maxmemory'.
- Possible causes:** Memory limit reached and eviction policy does not allow the command.

Check:
`INFO memory`
`CONFIG GET maxmemory`
`CONFIG GET maxmemory-policy`

Fix: Increase memory, change eviction policy, or remove data.

Verify: Command works again.

6. HIGH EVICTION RATE



- Symptom:** High number of evicted keys.
- Possible causes:** Memory pressure, low maxmemory, wrong eviction policy, large dataset.

Check:
`INFO stats | grep evicted_keys`
`INFO memory`
`CONFIG GET maxmemory-policy`

Fix: Increase memory, optimize data, adjust eviction policy (e.g. `allkeys-lru`).

Verify: Eviction rate decreases.

7. REPLICA FALLS BEHIND



- Symptom:** Replica lag is high.
- Possible causes:** High write load, slow replica, network latency, insufficient resources.

Check:
`INFO replication` (redis-cli on replica)
`master_link_status`
`master_last_io_seconds_ago`
`slave_repl_offset`

Fix: Improve network, increase resources, check for slow commands on replica.

Verify: Replica lag reduces.

8. PERSISTENCE FAILURE



- Symptom:** RDB or AOF not saving or loading correctly.
- Possible causes:** Disk full, permission issues, misconfiguration, corrupted files.

Check:
`INFO persistence`
`CONFIG GET dir`
`CONFIG GET dbfilename`
`ls -lh` (check disk space)
`redis.log` (check errors)

Fix: Free disk space, fix permissions, correct configuration, restore from backup if needed.

Verify: Check persistence status.

9. CONNECTION EXHAUSTION



- Symptom:** Too many connections or cannot accept new clients.
- Possible causes:** High traffic, connection leaks, low maxclients.

Check:
`INFO clients`
`CONFIG GET maxclients`
`ss -ant | grep 6379 | wc -l`

Fix: Increase maxclients, fix connection leaks, use connection pooling.

Verify: New connections succeed.

10. APPLICATION REPORTS TIMEOUTS



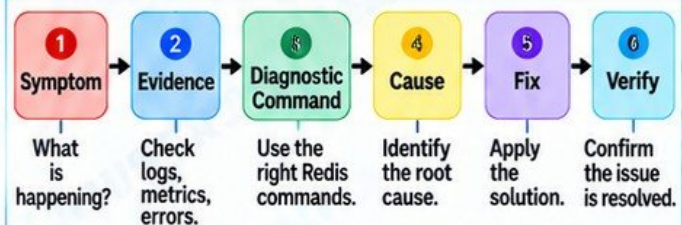
- Symptom:** Application experiences timeouts when accessing Redis.
- Possible causes:** High latency, network issues, resource exhaustion, blocking commands.

Check:
`SLOWLOG GET 10`
`INFO stats`
`INFO cpu`
`INFO memory`

Fix: Identify root cause (latency, CPU, memory, network), optimize and scale if required.

Verify: Application responds normally.

11. TROUBLESHOOTING FLOW



12. KEY TAKEAWAYS

- Start with the symptom and collect evidence.
- Use Redis commands to find the root cause.
- Consider application, network, and infrastructure.
- Make one change at a time.
- Verify the fix before closing the investigation.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

KNOWLEDGE
BUILDS OPPORTUNITY

Redis Data Loss, Backup, and Recovery

Protect your Redis data with the right backup strategy and be ready to recover when things go wrong.

1. WHAT CAN CAUSE DATA LOSS



- Server or VM/instance failure
- Disk failure or corruption
- Accidental data deletion (e.g. FLUSHALL)
- Misconfiguration
- Human error
- Software bugs
- Power outage or hardware issues
- Cloud provider or infrastructure failure
- Lack of proper backups

Data loss can happen at any time, plan for it before it happens.

2. PERSISTENCE IS NOT BACKUP



- RDB and AOF are persistence mechanisms, not backups.
- They help Redis survive restarts, but they are not a replacement for a proper backup strategy.
- They are stored on the same server and can be lost if the server or disk fails.
- Backups should be stored separately, preferably in another system or cloud storage.

Persistence helps you recover from restarts. Backups help you recover from disasters.

3. RDB BACKUP STRATEGY



- RDB (Redis Database) creates a point-in-time snapshot.
- Can be configured for automatic snapshots (save) or manual (BGSAVE).
- Lightweight and good for large datasets.
- Store RDB files in a separate backup location.
- Consider regular automated backups.

```
# Manual backup
redis-cli BGSAVE
# RDB file location (default)
/var/lib/redis/dump.rdb
```

4. AOF CONSIDERATIONS



- AOF (Append Only File) logs every write operation.
- Provides better durability than RDB.
- Can be larger in size.
- Use appendfsync settings (everysec is a good balance).
- AOF can be used for backup, but it is still stored on the same server, so copy it to external storage regularly.

```
# Example AOF configuration
appendonly yes
appendfsync everysec
```

5. BACKUP DESTINATIONS



- Store backups outside the Redis server.
- Use cloud storage (e.g. AWS S3, Azure Blob, Google Cloud Storage).
- Use a separate server or backup system.
- Keep multiple versions (retention policy).
- Consider encryption for sensitive data.
- Automate backups using scripts or cloud tools.

A backup in the same server or disk does not protect you from recover failure.

6. RESTORE PROCEDURE



- 1 Stop Redis (if needed).
- 2 Copy the backup file (RDB or AOF) from your backup location.
- 3 Place it in the correct directory (e.g. /var/lib/redis/).
- 4 Start Redis.
- 5 Verify the data.

```
# Example: restore from RDB
systemctl stop redis
cp dump.rdb /var/lib/redis/dump.rdb
systemctl start redis
redis-cli INFO keypace
```

7. TESTING RESTORES



- Regularly test your backups by restoring them in a non-production environment.
- Verify that the data is complete and usable.
- Test both RDB and AOF restores.
- Include restore testing in your disaster recovery plan.

A backup that has never been restored is not enough evidence of recoverability.

8. RPO AND RTO



- RPO (Recovery Point Objective): How much data you can afford to lose (time since last backup).
- RTO (Recovery Time Objective): How quickly you need to recover and restore service.
- Define RPO and RTO based on your business requirements.
- Your backup frequency and restore process should meet these objectives.

Be realistic and align RPO and RTO with business needs.

9. DISASTER RECOVERY THINKING



- Plan for regional failures (e.g. multi-region backups).
- Keep backups in different availability zones or regions.
- Document your recovery procedure.
- Regularly test your disaster recovery plan.
- Consider using Redis replication with backups for better resilience.

Think beyond a single server. Plan for disasters, not just failures.

10. PRODUCTION AWARENESS



- A backup that has never been restored is not enough evidence of recoverability.
- Automate backups and monitor them.
- Verify backup integrity.
- Have a clear and tested recovery plan.
- Data is important. Treat Redis backups as part of your production operations.

“ A backup that has never been restored is not enough evidence of recoverability.”



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

KNOWLEDGE
BUILDS OPPORTUNITY



Redis Troubleshooting Playbook

A practical junior engineer workflow to find and fix Redis problems quickly and confidently.

1. IS REDIS RUNNING?



- Check if the Redis process is running.
- Verify the service status (or container/pod).
- Check logs for startup errors.

```
# Linux (systemd)
sudo systemctl status redis
# Docker
docker ps | grep redis
# Kubernetes
kubectl get pods | grep redis
```

If Redis is not running, check logs and fix the startup issue first.

2. CAN THE CLIENT REACH THE HOST AND PORT?



- Check network connectivity.
- Verify host and port.
- Check firewalls and security groups.
- Ensure Redis is listening on the correct interface (usually 0.0.0.0 or specific IP).

```
# Test connection
nc -zv <host> <port>
telnet <host> <port>
# Check listening port
ss -tulpn | grep 6379
```

If you cannot reach the host and port, check network, firewall and security group rules.

3. IS AUTHENTICATION SUCCEEDING?



- Verify the password or ACL user.
- Check for authentication errors in logs.
- Make sure the client is using the correct credentials.

```
redis-cli -h <host> -p 6379
-a <password> PING
# ACL example
redis-cli ACL LIST
redis-cli -u redis://user:pass
@host:6379 PING
```

Authentication failures are often due to wrong credentials or missing ACL permissions.

4. ARE CONNECTIONS EXHAUSTED?



- Check the number of connected clients.
- Compare with maxclients.
- Look for many idle or stuck connections.

```
redis-cli INFO clients
redis-cli CLIENT LIST
```

If you have too many connections, increase maxclients or fix connection leaks in the application.

5. IS MEMORY EXHAUSTED?



- Check used memory and maxmemory.
- Look for memory fragmentation.
- Identify large keys.

```
redis-cli INFO memory
redis-cli MEMORY STATS
redis-cli --bigkeys
```

If memory is exhausted, consider increasing memory, optimizing key usage, or setting an eviction policy.

6. ARE EVICTIONS OCCURRING?



- Check eviction statistics.
- Understand your maxmemory policy (e.g. allkeys-lru).
- High evictions can cause cache misses and application issues.

```
redis-cli INFO stats | grep evicted
redis-cli INFO memory | grep
maxmemory_policy
```

Frequent evictions may indicate insufficient memory or a workload that does not fit in memory.

7. IS LATENCY INCREASING?



- Check latency metrics.
- Look for latency spikes and events.
- Identify slow commands or high CPU usage.

```
redis-cli LATENCY LATEST
redis-cli LATENCY HISTORY
redis-cli INFO stats
redis-cli SLOWLOG GET 10
```

High latency can be caused by slow commands, high CPU, memory pressure or network issues.

8. ARE SLOW COMMANDS PRESENT?



- Use SLOWLOG to find slow commands.
- Check command frequency and duration.
- Look for expensive operations (e.g. KEYS, LRANGE, SMEMBERS).

```
redis-cli SLOWLOG GET 10
redis-cli SLOWLOG LEN
redis-cli SLOWLOG RESET
```

Remove or optimize slow commands. Avoid using blocking commands in production.

9. IS PERSISTENCE HEALTHY?



- Check RDB and AOF status.
- Verify the last successful save.
- Look for persistence errors in logs.

```
redis-cli INFO persistence
ls -lh /var/lib/redis/
tail -n 50 /var/log/redis/redis.log
```

If persistence is failing, check disk space, file permissions and configuration.

10. IS REPLICATION HEALTHY?



- Check replication status on primary and replicas.
- Verify replica lag.
- Ensure replicas are connected and in sync.

```
redis-cli INFO replication
redis-cli INFO stats | grep
master_link_status
redis-cli INFO stats | grep
master_last_io_seconds_ago
```

If replication is unhealthy, check network, replication configuration and resource usage.

11. WHAT CHANGED RECENTLY?



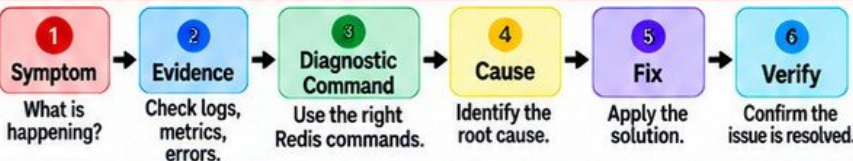
- Check recent deployments or configuration changes.
- Review logs and monitoring alerts.
- Look for infrastructure changes (instance type, network, security groups).
- Check application changes (e.g. new feature or version).

Recent changes often explain sudden issues. Always review the timeline.

12. USEFUL REDIS COMMANDS

Purpose	Command
Test connection	PING
Check server info	INFO
List clients	CLIENT LIST
Check slow commands	SLOWLOG GET 10
Check memory	MEMORY STATS
Check replication	INFO replication
Check persistence	INFO persistence
Check stats	INFO stats

13. TROUBLESHOOTING FLOW



14. FINAL RULE



Fix → Verify → Monitor

Do not stop at fixing the issue. Always verify the fix and keep monitoring to prevent it from happening again.



@veriqta



veriqta



veriqta



veriqta



veriqta



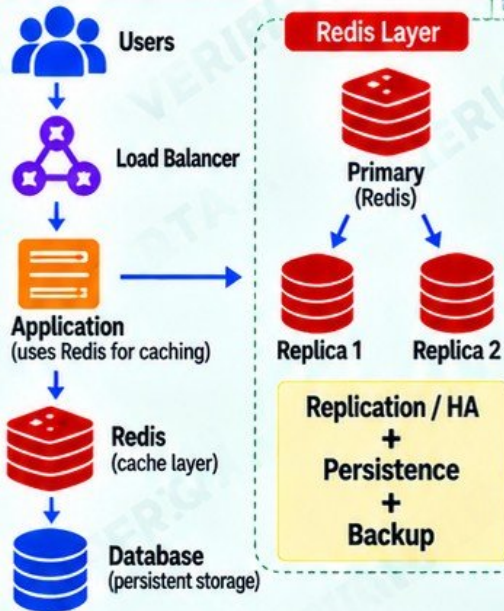
veriqta

KNOWLEDGE
BUILDS OPPORTUNITY

Redis Complete Production Project

Bring the entire handbook together and deploy Redis in a real production-style environment.

1. OVERALL ARCHITECTURE



2. MONITORING AND ALERTING



3. JUNIOR ENGINEER CONFIGURES

- 1 Redis deployment (Docker or Kubernetes)
- 2 Application caching (connect and use Redis)
- 3 TTL strategy (set appropriate expiration)
- 4 Memory limit and eviction policy
- 5 Authentication and ACLs (secure access)
- 6 Persistence (RDB and/or AOF)
- 7 Replication or high availability (Sentinel or Cluster)
- 8 Monitoring and alerts (Prometheus + Grafana)
- 9 Backup and restore (automate backups)
- 10 Failure testing (simulate real problems)
- 11 Troubleshooting (use the playbook)



Goal: Deploy a secure, highly available, monitored and recoverable Redis environment that can run in production.

4. EXAMPLE REDIS CONFIGURATION

```
# /etc/redis/redis.conf (key settings)
bind 0.0.0.0
port 6379
requirepass YourStrongPassword
maxmemory 2gb
maxmemory-policy allkeys-lru
appendonly yes
appendfsync everysec
save 900 1
save 300 10
save 60 10000
```

Adjust these settings based on your environment and workload requirements.

5. APPLICATION EXAMPLE (PYTHON)

```
import redis
r = redis.Redis(
    host='redis-host',
    port=6379,
    password='YourPassword',
    decode_responses=True
)

# Set data with TTL (1 hour)
r.setex('user:123', 3600, 'John')

# Get data
value = r.get('user:123')
print(value)
```

6. BACKUP EXAMPLE

```
# Manual RDB backup
redis-cli BGSAVE
cp /var/lib/redis/dump.rdb /backup/

# Backup to cloud storage (AWS S3 example)
aws s3 cp /backup/dump.rdb s3://my-bucket/redis/
```



Automate backups using cron jobs or a backup tool. Encrypt backups and store them offsite (e.g. S3, Azure Blob, GCS).

7. TESTING HIGH AVAILABILITY



- Set up Redis Sentinel or Cluster.
- Simulate a primary failure.
- Verify automatic failover.
- Ensure the application reconnects.
- Check data consistency.
- Monitor logs and metrics during failover.

8. MONITORING DASHBOARD



- Track memory usage.
- Track operations per second.
- Monitor cache hit ratio.
- Monitor latency and slow commands.
- Track replication lag.
- Set alerts for critical thresholds.

9. FAILURE TESTING IDEAS



- Stop the Redis service.
- Fill memory to trigger evictions.
- Create slow commands (e.g. KEYS *).
- Break replication (network issue).
- Corrupt persistence files (test recovery).
- Simulate connection exhaustion.

10. FINAL FAILURE LAB



- 1 Intentionally create a Redis problem (e.g. fill memory, stop service, or corrupt a file).
- 2 Investigate the evidence (logs, metrics, commands).
- 3 Identify the root cause.
- 4 Fix the issue.
- 5 Verify that the service has recovered.
- 6 Document what happened and what you learned.

11. PROJECT SUCCESS CRITERIA

- ✓ Users can access the application successfully.
- ✓ Redis is properly configured, secured and monitored.
- ✓ Application uses Redis caching with appropriate TTL.
- ✓ High availability is working (failover tested).
- ✓ Backups are automated and tested (restore works).
- ✓ Monitoring and alerts are in place.
- ✓ You can troubleshoot and recover from common failures.



@veriqta



veriqta



veriqta



veriqta



veriqta



veriqta

KNOWLEDGE
BUILDS OPPORTUNITY



SCALE YOUR SKILLS. BUILD YOUR FUTURE.